3×3 2048 自主游戏算法实验报告

蔡明辰　潘亮　陈达　孟维祎

摘要：针对 3×3 棋盘空间有限、随机落块容易导致死局的问题，本实验比较 MCTS、完整 Expectimax、TD 和 PPO 在 2048 中的表现，并考察三种 TD 价值表示。以终局数字总和为主要指标，在 20 个固定测试种子下，完整 Expectimax 的平均总和为 761.0，乐观 5-tuple TD 为 612.1，两者均达到 20 局生成 256 的结果。搜索方法能够直接利用环境规则，TD 则把经验保存在权重中，降低每步决策所需的计算。不同方法的训练量和搜索开销差异较大，因此本实验主要比较已有实现的效果，并讨论结果受计算预算和超参数影响的原因。

1　任务与环境

本实验要求智能体在 3×3 的 2048 棋盘上自主决策，使终局所有数字之和尽量大，同时尽可能生成不小于 256 的方块。动作包括左、右、上、下。每次有效移动后，环境从空格中均匀选取一个位置，以 0.9 和 0.1 的概率分别生成 2 和 4；没有合法动作时游戏结束。生成 256 后继续游戏，以避免提前终止影响总和比较。

合并只改变方块的分布，不改变总和。若以 S_t 表示第 t 步的棋盘总和，基础奖励设为 $r_t = S_{t+1} - S_t$，有效移动的奖励即为新落块值 2 或 4。提高终局总和需要让棋盘保持可移动，从而容纳更多新方块。

实现上保留原环境的移动、合并与显示功能，通过继承环境类调整棋盘大小、随机源、奖励和统计信息。算法分别实现动作选择接口，配置读取、权重加载和评测由公共入口完成。这样可以在沿用原有代码结构的同时，复用相同规则比较不同策略。

2　方法

2.1　MCTS 与完整 Expectimax

MCTS 通过反复模拟估计动作价值。搜索树使用 UCT 选择动作：

$$Q(s,a) + c\sqrt{\frac{\ln(N(s)+1)}{N(s,a)}}, \tag{1}$$

其中 $N(s)$ 和 $N(s,a)$ 分别为状态与动作的访问次数，c 控制探索强度。模拟显式考虑随机落块，达到设定深度后，用棋盘总和、空格数、蛇形排列和达标奖励估值，再向根节点回传。其优点是无需离线训练，但有限模拟次数会带来较大的估计误差。

Expectimax 利用已知落块概率求期望。玩家节点选择价值最大的动作，机会节点枚举空格以及出生块 2 和 4。记 x 为移动后、落块前的后状态，$A(s)$ 为合法动作集合，$E(x)$ 为空格集合，有

$$\begin{aligned} V(s) &= \max_{a \in A(s)} C(\text{move}(s,a)), \\ C(x) &= \frac{1}{|E(x)|} \sum_{i \in E(x)} [0.9V(x \oplus 2_i) + 0.1V(x \oplus 4_i)]. \end{aligned} \tag{2}$$

本实验使用完整搜索，不设置有限深度的启发式截断，终局效用为

$$U(s) = S(s) + 256\,\mathbf{1}\{\max(s) \geq 256\}. \tag{3}$$

因此，算法优化的是期望总和与达标奖励的加权目标。求解器用 C++ 实现，借助棋盘编码、旋转与镜像对称以及缓存减少重复求解。完整搜索有较高的首次求解和存储成本。

2.2　TD 价值学习

TD 根据相邻状态的价值差更新估计。使用后状态可以把玩家移动与随机落块分开：对每个合法动作，先计算移动后的棋盘，再比较其长期价值。以未来总和增量为价值时，TD(0) 误差为

$$\delta = z + \gamma V(x') - V(x), \tag{4}$$

其中 z 为新落块值，终局后续价值为 0。乐观 5-tuple 采用 $\gamma = 1$。CNN 和普通 n-tuple 的具体实现使用包含棋盘总和与达标项的效用或残差价值，不能直接把三者的更新目标视为完全相同。

表 1　三种 TD 价值表示

方法	价值表示	主要特点
乐观 5-tuple	3 张 16^5 权重表，24 个对称特征	初始总价值设为 300，以偏高估值鼓励探索 [1]
CNN-TD	11 个 one-hot 输入平面，79,874 个参数	学习棋盘效用残差，使用 Adam 与梯度裁剪
n-tuple TD	行、列、2×2 局部模式	对称规范化，结合启发式先验与表项更新

乐观初始化的作用在于：较少访问的局面仍保留较高估计，智能体有动力尝试这些局面；随着训练，估计逐渐接近实际回报。局部模式表则通过共享相似棋盘的特征，提高经验利用率 [1, 2]。

2.3　PPO 策略学习

PPO 直接学习动作概率，同时训练价值网络，通过 GAE 估计优势，并用剪切概率比限制一次更新的幅度 [3]。本实验采用卷积残差网络，在输出端屏蔽非法动作。评测时冻结权重并选择概率最大的合法动作。与查表 TD 相比，PPO 依赖神经网络训练，其效果还受样本量、奖励尺度和折扣因子影响。

3　实验设置与超参数

各方法使用种子 10000 至 10019 进行 20 局测试，运行到自然终局，不设置测试步数上限。测试期间不更新权重，搜索随机数与环境落块随机数分开。相同种子能够统一随机数来源，但策略改变后，棋盘空格不同，实际落块位置也会不同。

表2列出实际使用的预算与主要参数。训练局数、环境步数、搜索模拟次数分别对应不同开销，不能简单等同。学习方法的参数以模型保存的记录为依据。

表 2　计算预算与主要超参数

方法	预算	主要参数
MCTS	每步 128 次模拟	深度 12，$c = 70$，随机模拟比例 0.15，空格权重 18
完整 Expectimax	搜索至终局并缓存	无深度截断，终局达标奖励 256
乐观 5-tuple TD	300,000 局	训练种子 42，初始总价值 300，24 个特征
CNN-TD	50,000 局	学习率 3×10^{-4}，批量 1024，探索率 0.10 降至 0.01
n-tuple TD	5,000 局	$\alpha = 0.2$，探索率 0.20 降至 0.02，空格权重 18
PPO	50,176 个 RL 环境步	学习率 10^{-4}，$\gamma = 0.999$，GAE $\lambda = 0.95$，clip=0.2

CNN 和 PPO 已有模型的学习率分别为 3×10^{-4} 和 10^{-4}，PPO 折扣因子为 0.999，与当前训练入口的默认值存在差别。PPO 还存在教师预训练记录，因此 50,176 步仅表示强化学习阶段的计数。最终榜单中不同 TD 模型的训练量、表示方式和先验同时变化，不作单因素消融。下面使用另有记录的历史对照讨论参数影响。

3.1 超参数对照与讨论

为讨论参数影响，另整理早期探索记录。这部分与后文 20 局最终榜单分开：表3各模型均从头训练 10 万局，训练种子为 42，测试种子为 20000 至 20499，共 500 局；前 70% 训练使用 $\alpha = 0.1$，之后为 0.03，前 80% 使用 $\epsilon = 0.02$，之后为 0.002。训练局数和参数日程一致，但不同策略的交互步数与耗时仍有差异。

表 3　历史 TD 对照：相同训练局数下的参数与结构变化

设置	平均总和	达到 256	相对基准的变化
对称 5-tuple，初值 0（基准）	605.76	89.0%	—
5-tuple，不共享对称	417.69	72.2%	对称特征
对称 4-tuple	432.88	74.0%	局部模式结构
对称 5-tuple，初值 300	540.78	94.4%	乐观初始化
对称 5-tuple，合并分奖励	592.64	88.4%	奖励目标及尺度
对称 5-tuple，首次 256 加奖	482.92	86.8%	奖励增加达标项
对称 6-tuple，首次 256 加奖	401.52	66.6%	对比上一行的模式

表示与初始化。对称共享使平均总和由 417.69 提高到 605.76；它使旋转、镜像局面共享经验，是本轮较明确的改进。4-tuple 低于 5-tuple，而采用达标奖励的 6-tuple 也低于同奖励的 5-tuple。由于模式数量和表容量同时变化，这说明当前训练量下更大的局部模式未必更好，不能单独归因于特征长度。初值 300 将达标率从 89.0% 提高到 94.4%，平均总和却降低约 65，说明更容易达到 256 与继续累积总和并不总是一致；现有记录没有比较多个非零初值，不能认定 300 最优。

训练量、学习率与探索率。表4比较 10 万局与另一次从头训练 30 万局的结果，测试仍使用同一组 500 个种子。三组平均总和均上升，但乐观版本的达标率下降，更长训练没有让两项指标同步提高。两轮的退火时间点也随总预算后移，不能视为同一训练轨迹上的两个检查点。

表 4　训练预算变化：平均总和与达标率

设置	10 万局总和	30 万局总和	10 万局达标	30 万局达标
普通对称 5-tuple	605.76	629.84	89.0%	93.0%
乐观对称 5-tuple	540.78	618.68	94.4%	90.4%
5-tuple 加达标奖励	482.92	595.00	86.8%	90.6%

训练日志中降低学习率与探索率后回报上升，但日志包含探索动作，又与训练进度重合，无法分离各因素的贡献。在另一组从同一权重续训 20 万局的对照中，固定 $\alpha = 0.01$，$\epsilon = 0.0005$ 与 0 分别取得 708.79 和 699.23 的平均总和；300 局配对 bootstrap 的差值 95% 区间为 $[-16.4, 35.3]$，尚不足以确认保留微量探索的优势。

前瞻深度与计算成本。固定 10 万局普通 5-tuple 权重，只改变决策时的前瞻深度，结果见表5。深度 3 相对深度 2 提高约 6.7% 的总和，耗时约为 14.4 倍。该结果支持浅层搜索与 TD 估值结合；计时来自历史编译实现，排除了编译预热，不能代表当前 Python 入口速度或完整 Expectimax 的成本。

结论边界。现有证据支持对称共享，以及在允许更多决策时间时加深前瞻。乐观初始化和达标奖励需要同时观察总和与达标率；合并分奖励还改变了奖励尺度，不能只解释为目标差异。学习率、MCTS 的模拟次数与探索系数、PPO 的 clip 和折扣因子都缺少逐项多取值对照，目前只能说明所用配置及其作用。以上每种配置只有一个训练种子，评测置信区间也不包含重新训练的波动，不能称为全局最优超参数。

表 5　固定 TD 权重的搜索深度对照（500 局）

决策方式	平均总和	达到 256	每步耗时/ms
直接 TD 估值（深度 1）	605.76	89.0%	0.00076
TD 与深度 2 搜索	670.88	94.2%	0.00615
TD 与深度 3 搜索	716.07	95.8%	0.08855

4　结果与分析

表6给出平均终局总和、生成 256 的比例和平均步数。MCTS、完整 Expectimax、乐观 TD 及 PPO 的数据来自逐局记录，CNN 与普通 n-tuple 的数据来自同一 20 局协议的汇总结果。

表 6　20 局测试结果

方法	平均总和	达到 256	平均步数
完整 Expectimax	761.00	20/20（100%）	344.55
乐观 5-tuple TD	612.10	20/20（100%）	277.60
CNN-TD	418.80	16/20（80%）	188.50
n-tuple TD	384.00	14/20（70%）	172.15
MCTS	266.00	6/20（30%）	119.45
PPO	211.40	2/20（10%）	94.55

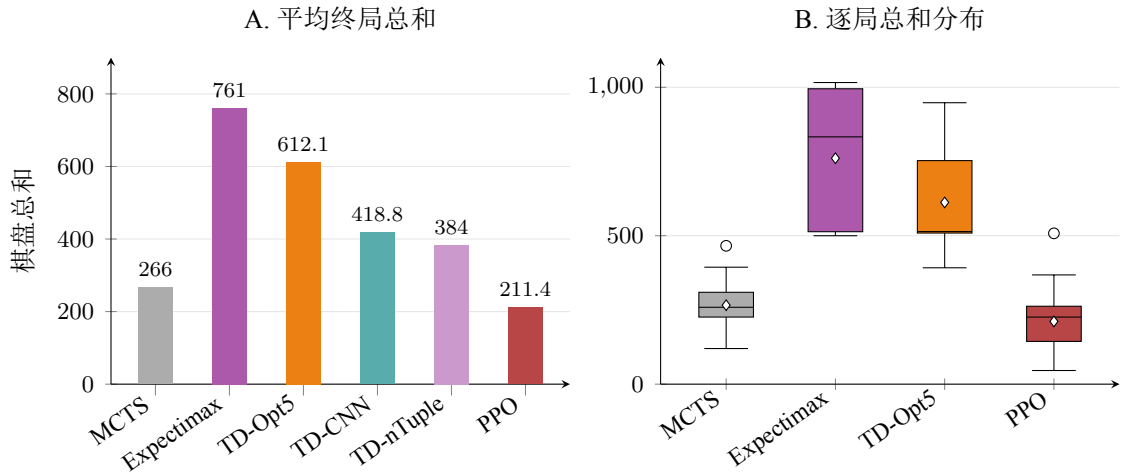


图 1　平均终局总和与逐局分布。右图仅包含有逐局记录的四种方法，箱体表示四分位范围，横线为中位数，菱形为均值。

完整 Expectimax 的平均总和最高，比乐观 TD 高 148.9，约 24.3%。两者均在 20 局中生成 256，完整搜索在较高总和局面上的收益更明显。乐观 TD 排名第二，推理主要是特征查表，适合在线计算资源有限的场景。CNN 与普通 n-tuple 都超过当前 MCTS，但由于训练量与先验不同，尚不能据此判断某种表示本身更优。

MCTS 的结果说明，小预算随机模拟配合启发式估值，在 3×3 的紧凑棋盘上仍容易陷入死局。PPO 的平均总和最低，当前训练量与奖励设置下尚未形成稳定策略。其原因需要进一步训练和对照实验确认，不能仅凭这组结果归因于 PPO 算法本身。平均步数与总和总体一致，这是因为活得更久通常能够接纳更多出生块。

20 局全达标仍是有限样本结果。若把各局视为独立同分布试验，20/20 对应的 95% Wilson 区间约为 [83.9%，100%]。在这 20 个种子上进行 20,000 次配对 bootstrap，Expectimax 与乐观 TD 平均总和差的 95% 区间约为 [23.6，273.0]，该区间不包含重新训练带来的波动。另一次乐

观 TD 的 500 局测试平均总和为 639.408，463 局生成 256；它使用不同测试设置，不与表6合并。

5 总结

在本实验的 3×3 环境中，完整 Expectimax 取得最高平均总和，乐观 5-tuple TD 则兼顾效果与在线决策开销。搜索能充分利用已知转移规则，TD 可以把搜索或交互经验转化为快速估值，二者各有适用条件。后续可在共同奖励目标下统一运行时间或每步时限，分别记录训练、建表、内存与推理开销，并用多个训练种子和新的测试种子重复实验。对学习率、探索率和搜索预算逐项调整，才能更清楚地区分参数设置与算法结构的影响。

6 小组分工

成员	负责内容
蔡明辰	环境适配、随机种子与规则测试，统筹代码整合
潘亮	TD 方法实现、模型训练与超参数整理
陈达	MCTS、Expectimax 和 PPO 的方法实现及对照分析
孟维祎	评测数据整理、图表制作与报告撰写

参考文献

[1] Guei H, Chen L-P, Wu I-C. Optimistic Temporal Difference Learning for 2048. 2021.
[2] Szubert M, Jaskowski W. Temporal Difference Learning of N-Tuple Networks for the Game 2048. CIG, 2014.
[3] Schulman J, et al. Proximal Policy Optimization Algorithms. 2017.